

El cheat sheet definitivo de CLAUDE CODE para programadores

Qué es

- Corre en terminal, escritorio e IDE
- Actúa sobre tu repo: lee → planifica → edita → ejecuta
- Se extiende con: CLAUDE.md • skills • subagentes • slash commands • hooks • MCP

Instalación y auth

- Instalar: `npm install -g @anthropic-ai/claude-code` (Node 18+)
- Arrancar: `cd tu-proyecto && claude`
- Login: `/login` (cuenta de Claude, vía navegador)
- Con API key: variable `ANTHROPIC_API_KEY`
- Empresa: Bedrock (`CLAUDE_CODE_USE_BEDROCK`) o Vertex (`CLAUDE_CODE_USE_VERTEX`)

Flujo recomendado

- **Explorar** → "lee `/src/auth` y explícame cómo funciona"
- **Planificar** → `/plan` o `Shift+Tab`; cierra con "no implementes todavía"
- **Implementar** → ejecutar + escribir tests
- **Confirmar** → correr tests, arreglar, commit
- Tarea chica y clara → directo, sin plan

Manejar el contexto

- **Regla:** no pasar de la mitad de la ventana
- `/context` → ver cuánto usas
- `/compact` → liberar espacio (antes de llenarte)
- `/clear` → tarea nueva, contexto limpio
- Truco: volcar a un `.md` → `/clear` → releer

Permisos (settings.json)

- **allow** → lo que puede hacer solo
- **ask** → pide confirmación (ej. `git push`, instalar deps)
- **deny** → prohibido: `.env`, secretos, credenciales

Comandos del día a día

Comando	Qué hace
<code>/help</code>	Lista real de comandos de tu versión
<code>/init</code>	Genera el CLAUDE.md
<code>/clear</code>	Contexto limpio para una tarea nueva
<code>/compact</code>	Comprime la conversación
<code>/context</code>	Uso de contexto
<code>/plan</code>	Piensa y propone antes de codear
<code>/review</code>	Revisa tus cambios buscando bugs
<code>/model</code>	Cambia el modelo
<code>/rewind</code>	Vuelve atrás a un punto guardado
<code>/diff</code>	Visor de cambios
<code>/usage</code>	Límites y gasto

Atajos

Atajo	Qué hace
<code>Shift+Tab</code>	Cicla modos: Normal → Auto-aceptar → Plan
<code>!</code>	Corre un comando de shell
<code>@</code>	Menciona un archivo o carpeta
<code>Esc</code>	Interrumpe a Claude
<code>Esc Esc</code>	Menú de rewind
<code>Ctrl+B</code>	Manda un proceso largo al fondo

CLAUDE.md (memoria del proyecto)

- Se genera con `/init`
- Entre 60 y 200 líneas
- Lo más importante arriba
- Pídele que actualice su CLAUDE.md tras cada error

Que Claude verifique su propio trabajo

- `/verify` → confirma que el cambio funciona
- `/run` → levanta y prueba la app
- Pídele que escriba y corra los tests

Extensiones: quién las invoca

Pieza	Qué es	Quién la invoca
Slash command	Atajo con un prompt guardado	Tú (<code>/nombre</code>)
Skill	Instrucciones reutilizables	Claude, solo
Subagente	Asistente con contexto propio	Claude o tú
Hook	Script disparado por un evento	Automático
MCP	Conexión a herramientas externas	Configurado una vez

- Subagentes → `.claude/agents/`, contexto aislado, sus propias tools
- Hooks → auto-formato, logs, checks (corren shell: revisa lo que agregas)
- Plugins → empaquetan todo lo anterior

MCP: agregar y compartir

- Agregar: `claude mcp add <nombre> ...`
- Local (default) → solo tú, este proyecto (`~/.claude.json`)
- `--scope user` → todos tus proyectos
- `--scope project` → equipo: escribe `.mcp.json` (commit a git)
- Gestionar y autenticar: `/mcp` dentro de la sesión
- Para empezar: Playwright, Postgres, GitHub, Figma, Slack

Paralelo y automatización

- Worktrees → varias sesiones aisladas a la vez
- `Ctrl+B` → proceso al fondo
- `claude -p` → modo headless (CI, pre-commit)

Si algo no anda

- `/doctor` → diagnostica instalación y config
- `/mcp` → estado de los servidores (¿caído? reautenticar)
- `/permissions` → revisar permisos otorgados
- `--safe-mode` → arranca sin CLAUDE.md, skills, MCP ni hooks (aisla el problema)

Guía de decisión (cuándo usar qué)

- **CLAUDE.md** → memoria
- **Skills** → workflows repetibles
- **Subagentes** → aislamiento
- **Hooks** → control automático
- **MCP** → integraciones externas
- **Plugins** → empaquetar y reusar
- **Plan** → cambios grandes • Directo → cambios chicos

Errores comunes

- Contexto lleno sin `/compact` ni `/clear`
- CLAUDE.md gigante (se ignora)
- Saltarse el plan en cambios grandes
- No darle forma de verificar
- Confiar el rewind para cambios de shell (no se rastrean; no reemplaza Git)

Aprende a programar con AI en la carrera de **Full Stack AI**

AGENDA LLAMADA CON UN ASESOR AQUÍ